# Leverage Codex and LiteLLM to route every request to the right OCI model

Point Codex at a LiteLLM AI Gateway and every request it makes goes to the right one of twenty OCI Generative AI models, chosen by a router trained on judged outcomes; the same Codex builds and runs the gateway from a specification

Federico Kamelhar · Distinguished Architect, Agentic AI

2026-10-15

## Table of Contents

### The obstacle, and the shift

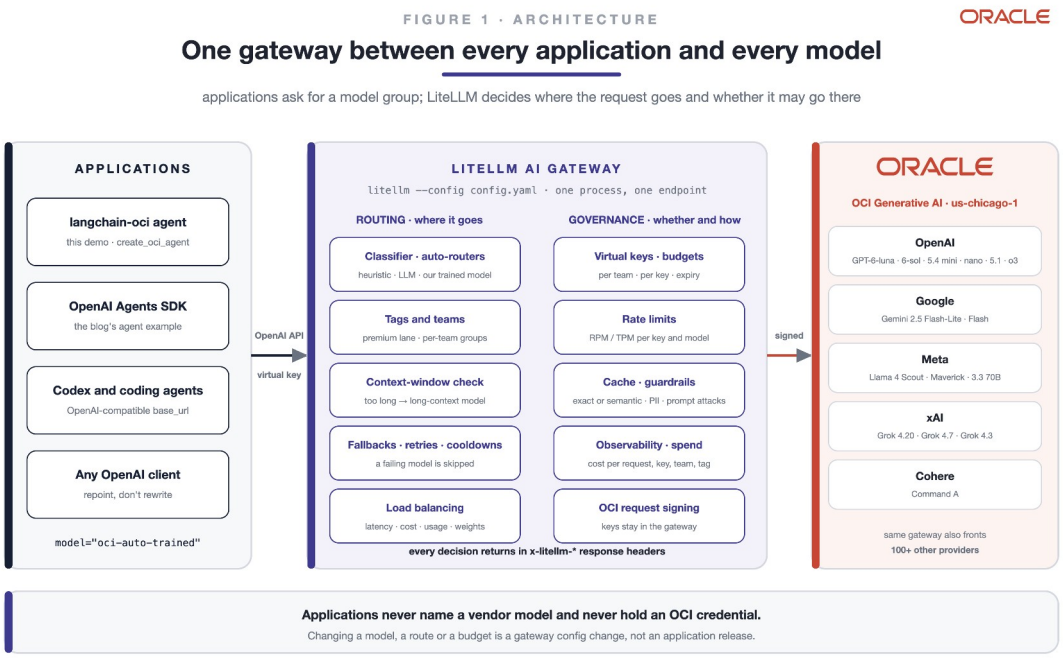
Ask a team why its agents cost more than they should, or answer worse than they could, and the honest answer is rarely the models. It is that every request goes to the same one. A password-reset question is answered well by the cheapest model in the catalog. A plan under constraints needs a reasoning model; code belongs with a coding model, an invoice photo with a vision model, a Japanese request with a multilingual one. Choosing per request is easy to say and hard to build: someone has to measure which model is good enough for what, wire the choice into a gateway, keep keys away from the agents, and run all of it in a cloud account without a credential leaking on the way.

This write-up shows that choice made per request, on Oracle Cloud Infrastructure (OCI), and then removes the part that makes teams hesitate: building it. The open-source LiteLLM AI Gateway puts twenty OCI Generative AI models, from OpenAI, Google, Meta, xAI and Cohere, behind one OpenAI-compatible endpoint. A small router, trained on answers a judge actually graded, decides how much model each request needs. LiteLLM's own configuration adds the business rules: a premium lane by tag, virtual keys with budgets, fallbacks, a context-window check. The agent sending the requests runs in an NVIDIA OpenShell sandbox and holds no key.

The person who builds and runs it talks to OpenAI Codex. The repository ships what Codex needs: the rules it loads when it starts, five skills, a build specification with checkpoints, and the prompts. Codex runs the OCI, Kubernetes and LiteLLM steps through the repository's scripts and the checks after them, and reports. The command lines still exist; they are what Codex runs underneath, not what the operator types.

One rule runs through everything: no secret passes through an agent, neither the one in the sandbox nor the one building the system.

**Watch it happen** — eight recorded beats of the running demo ship beside this write-up (`videos/`): a billing dispute with every call and the sandbox console, the premium lane, an invoice photo turned into a plan, the escape attempt, a failing deployment, a Japanese request, a race condition fixed on GPT-5.3-codex, and the three routers compared.




*One gateway between every application and every model. Applications ask LiteLLM for a model group with a virtual key; LiteLLM decides the route, applies the governance, signs the call and forwards it to OCI Generative AI. Applications never name a vendor model and never hold an OCI credential.*

## Codex, routed: one setting, the right model per request

The most useful client of this gateway turned out to be the coding agent people already run. Codex speaks the Responses API, and LiteLLM serves it, so one setting makes LiteLLM Codex's model provider and `oci-auto-trained` its model:

```
model_provider = "litellm"
model = "oci-auto-trained"

[model_providers.litellm]
name = "LiteLLM on OCI"
base_url = "http://127.0.0.1:4000/v1"     # or the cloud gateway's https://<host>/v1
env_key = "LITELLM_API_KEY"               # a LiteLLM virtual key, with its own budget
wire_api = "responses"
```

From then on every request Codex makes is routed. A question is answered by a fast model; a code fix goes to GPT-5.3-codex on OCI; Codex holds a LiteLLM key with a budget and no OCI or OpenAI credential. The sample ships a check that drives real Codex sessions and judges them from the outside, by the gateway's own log and by the result (`scripts/codex_e2e.py`, `evidence/codex/codex-e2e-2026-10-15.md`):

| You ask Codex | The gateway chose | Cost | Checked by |
|---|---|---|---|
| "What does HTTP status 429 mean?" | GPT-6-luna or GPT-5.4-nano, the fast route | about $0.001 | the answer names 429 |
| "Fix the data race in main.go" | GPT-5.3-codex, all four turns, no fallback | about $0.07 in all | `go vet` and `go run -race` print 1000 |
| the same question, in Spanish | GPT-6-luna, chosen by the trained router | about $0.001 | the answer, in Spanish |

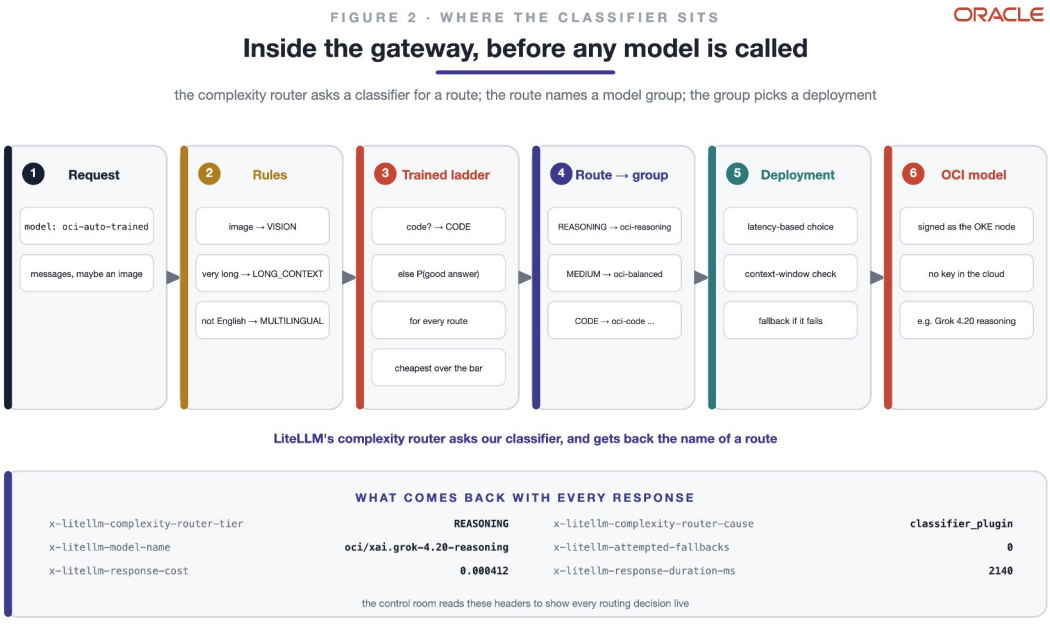
The table is from a fresh clone of the sample, with a clean Codex home that holds only the setting above (`evidence/codex/codex-clean-home-2026-10-20.md`); the repository's Codex check passed six of

six there, with no fallback. A Codex turn carries its whole context, instructions, tool definitions and conversation: about 9,000 tokens with that setting alone, about 50,000 with plugins and MCP servers. So a code turn costs about twenty times a question, and without routing every question is billed like one; the bigger the Codex setup, the more routing saves. One caveat the measurement found: on the fast route, the usage LiteLLM records for Codex's streamed calls has no input tokens, so the logged cost and the key's budget count only the output there.

Getting there took five fixes, each a quiet failure first: OCI's compressed event stream broke every streamed call; the GPT-5.3-codex provider could not stream, so every Codex turn fell back to GPT-5.1; Codex's own context, sent as user turns, made the router send every task to the strong route; OCI's chat API refused Codex's reasoning setting together with tools; and Cohere Command A answered Codex's 21-tool request with an empty reply. They are in the field notes below, and in the code.

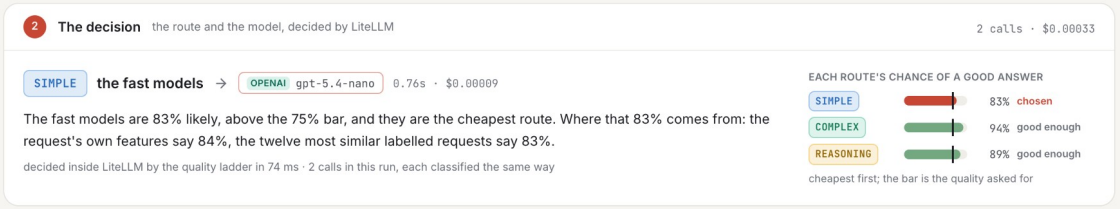
## What a request goes through

The application asks for `oci-auto-trained`, a model group, not a model. Inside the gateway and before any model is called, three layers decide where it goes, in order. Facts in the request come first, by rules: an image goes to VISION, more than about 30,000 tokens to LONG_CONTEXT, a language other than English to MULTILINGUAL. What the request needs comes second, by the trained router: a code check sends programming requests to CODE, and otherwise the router predicts, for each route, the chance that its answer is good enough, and takes the cheapest route over a quality bar, 75% unless the request asks for its own. Business rules come last, by LiteLLM: tags, keys and budgets, latency-based load balancing, fallbacks, retries and cooldowns.




*Inside the gateway, before any model is called. LiteLLM's complexity router asks the trained classifier for a route; the route names a model group; the group picks a deployment. Every decision returns in `x-litellm-*` response headers, so the application, and the control room, can read it.*

The decision is not a black box. The gateway returns it with the response, and the control room shows it the way an operator needs it: the route and the model, the reason in one sentence, and each route's chance of a good answer against the bar.



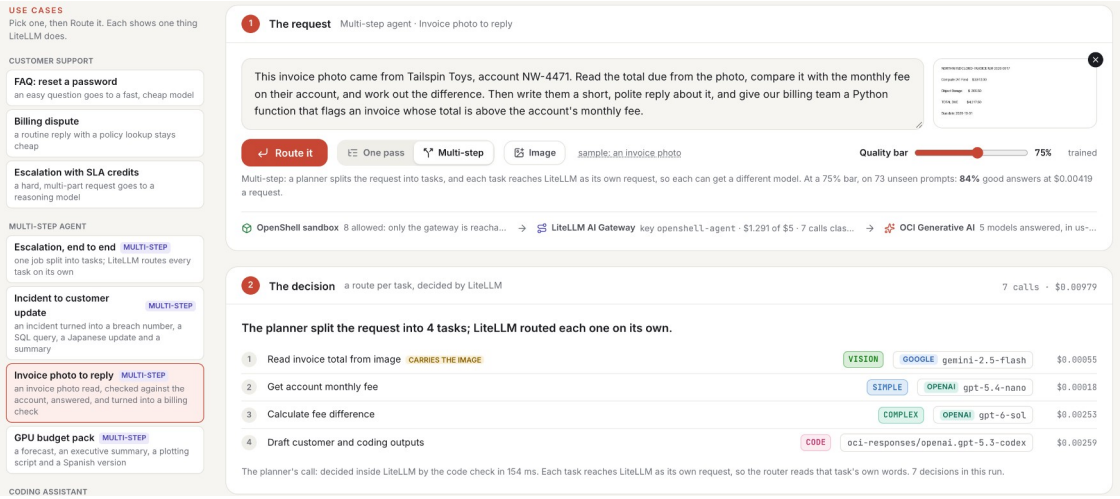

*One request's decision in the control room. The fast models were 83% likely to answer well, above the 75% bar, and they are the cheapest route, so GPT-5.4-nano answered in 0.76 seconds for $0.00009. The classifier decided in 74 ms.*

The same decision is in the logs, one line from the classifier and one from LiteLLM per request, which the control room streams live. These two come from the race-condition request, wrapped for the page:

```
classifier route=CODE criterion="model: code" CODE=0.61 SIMPLE=0.21 MEDIUM=0.11
           REASONING=0.07 102ms
litellm    key=openshell-agent asked=oci-auto-trained route=CODE why="model: code"
           model=openai.gpt-5.3-codex 5.77s $0.006454 tokens=304/423 fallbacks=0
```

The beats of the demo each show one thing LiteLLM does. Through the public control room on 13 October, after the fixes described below (`evidence/e2e-cloud-2026-10-13.txt`): a billing dispute stayed on the fast route for a fraction of a cent across two or three calls with tool lookups; the same request with a `premium` tag took the GPT-5.2 lane with no classifier at all; the deployment behind `oci-flaky` failed and LiteLLM answered from the balanced route instead; the Japanese request went to Cohere Command A by rule, a tool call and then the answer; the race condition went to GPT-5.3-codex on OCI's Responses API. Compared side by side on that coding request, LiteLLM's heuristic and its LLM classifier both chose the fast models, because they know four complexity tiers and no others; only the trained router sent it to CODE.

One job often holds several tasks, and an agent can split it. In plan mode a planner call writes the steps, and each step reaches the gateway as its own request, so each gets its own route:



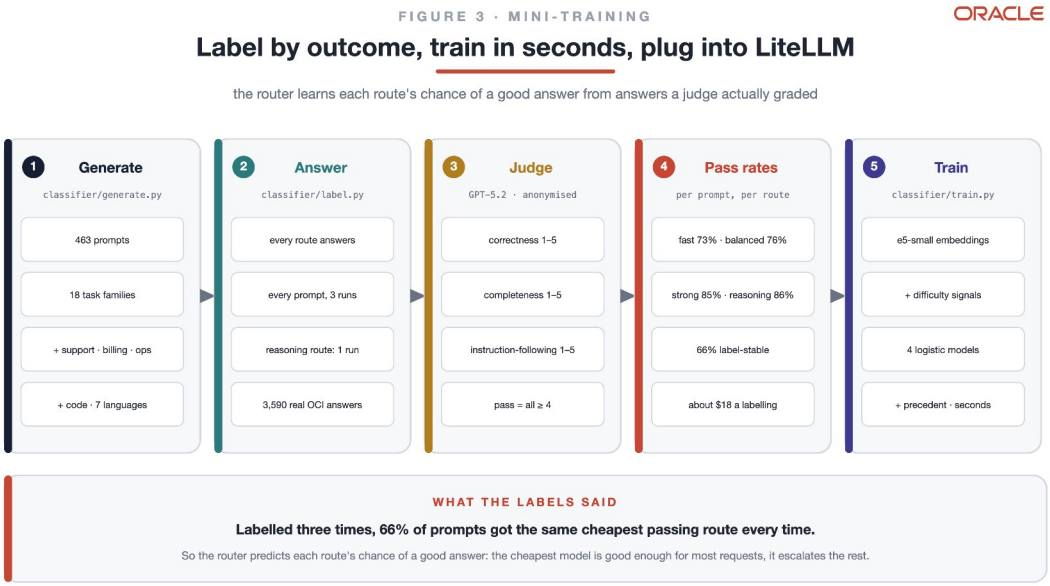

*An invoice photo, a fee lookup, a calculation, and a reply with a Python check: four tasks, four routes, four models. The photo went to VISION on Gemini 2.5 Flash, the lookup to the fast route on GPT-5.4-nano, the arithmetic to the strong route on GPT-6-sol, and the code to GPT-5.3-codex; seven calls, under a cent.*

## How the router was trained

The router learns from outcomes, not from anyone's opinion of how hard a prompt is. GPT-5.2 wrote 463 requests across 26 task families, from one-line facts to planning under constraints, including the support,

billing, IT operations, finance and people-operations requests a business sends, code, and seven languages; GPT-5.5 later added 128 in business teams' own words, coding requests and requests that only look like code. Every route answered every request through the gateway, three times, and the same judge graded the anonymised answers for correctness, completeness and instruction-following. An answer passes at 4 of 5 or better on all three. That is 3,590 real OCI answers, about $18 a labelling.

The labels taught the first lesson. Labelled three times, only 66% of the requests got the same cheapest passing route every time, so a router that predicts one label is chasing noise. This one predicts each route's chance of a good answer instead: per route, a logistic model on frozen multilingual-e5-small embeddings and seven difficulty signals, averaged with the pass rates of the twelve most similar labelled requests, and a code head that sends programming requests to CODE. Cross-validation inside the training split chose the routes it climbs, fast, strong and reasoning, and the 75% bar. Training takes 0.18 seconds after a 3.7-second embedding pass; the shipped model is about 220 KB and classifies in tens of milliseconds on one Ampere core (median 79 ms, `evidence/oke-run.json`).




*Label by outcome, train in seconds, plug into LiteLLM. The router learns each route's chance of a good answer from answers a judge actually graded.*

Does it pay? On 73 held-out requests never used in training, every answer a real OCI call graded by the same judge (`evidence/classifier-eval.json`), the trained router gave 84% good answers for $0.306, matching LiteLLM's LLM classifier (84%, $0.338) for 9% less and staying 5 points under always using the reasoning models (89%, $0.376) for 19% less. LiteLLM's heuristic cost the least among the routers, $0.082, at 79%. Only the trained router sent code and other languages to their own routes: 24 of 24 held-out requests, against none for the other two. On 150 fresh requests written by another model and graded by two judges from different vendors, the order held (`evidence/robustness.json`). It is a prediction, and it can be wrong on a single request: on two demo prompts it picks the strong route where the reasoning route passed more often (`evidence/strong-vs-reasoning.json`). The checks report those as known misses instead of hiding them.

## How the pieces fit

Each piece is small, and most of it is configuration. Routes are LiteLLM model groups in `gateway/config.yaml`, each deployment with its price, from OCI's public price list where it has one

(LiteLLM's built-in map undercounts OCI's Gemini 2.5 Flash about four times and does not list the newest models). The router the application asks for is one more entry:

```yaml
- model_name: oci-auto-trained              # what the application asks for
  litellm_params:
    model: auto_router/complexity_router
    complexity_router_config:
      classifier_type: custom
      classifier_plugin: trained_classifier.plugin
      fallback_tier: MEDIUM                 # if the classifier is slow or down
      tiers: {SIMPLE: oci-fast, MEDIUM: oci-balanced, COMPLEX: oci-strong,
              REASONING: oci-reasoning, CODE: oci-code, VISION: oci-vision,
              LONG_CONTEXT: oci-long, MULTILINGUAL: oci-multilingual}
```

The classifier is a plugin: an object with one async method that returns a route name, or nothing to decline. Ours asks a classifier service on the cluster with a three-second budget, and a classifier outage degrades routing, never availability: with the classifier scaled to zero, the same request answered on the balanced route, routing came back with the classifier, and a killed classifier pod was replaced in 20 seconds (`evidence/oke-run.json`).

OpenAI's GPT-5.3-codex is served by OCI only on its OpenAI-compatible Responses API, which LiteLLM's OCI provider does not speak. A LiteLLM custom provider, `gateway/oci_responses.py`, translates chat requests to Responses requests and the answers back, tool calls included, signed like every other OCI call; one line of `custom_provider_map` plugs it in behind the CODE route.

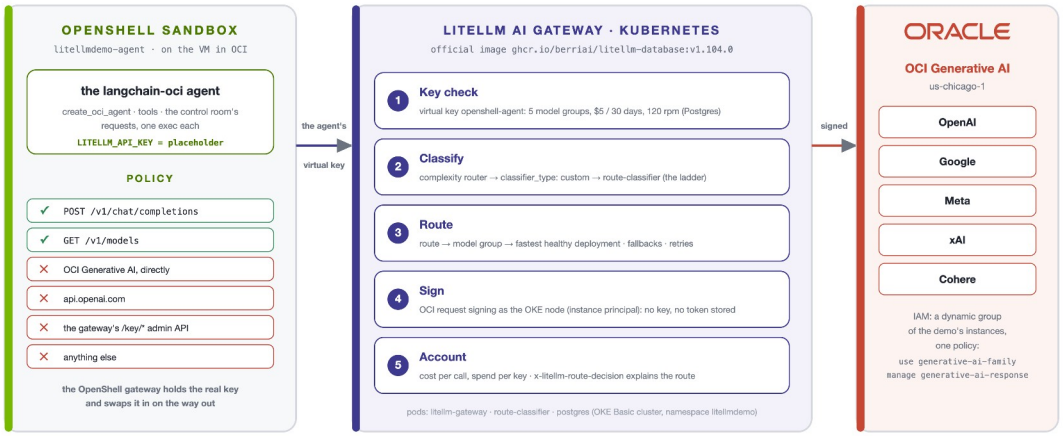
The agent is a langchain-oci agent whose model is an OpenAI-compatible client pointed at the gateway, asking for a model group. It runs in an OpenShell sandbox, and what it finds in its key variable is a placeholder: the OpenShell proxy swaps in the agent's LiteLLM virtual key on requests to the gateway, and nowhere else. The provider profile is the sandbox's whole network surface, two methods on one host for one binary. The key itself belongs to LiteLLM: five model groups, $5 every 30 days and a rate limit, kept in Postgres and charged per call.

FIGURE 4 · ISOLATION

ORACLE

**The agent holds nothing; LiteLLM decides, signs and accounts**

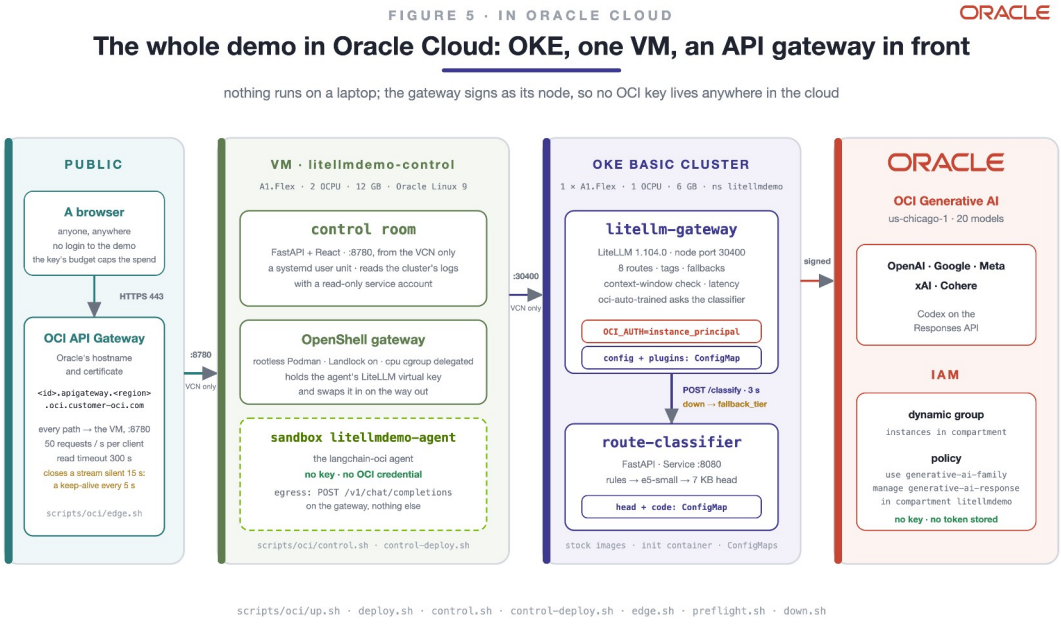
an OpenShell sandbox can reach one endpoint; every key lives one layer away from the code that uses it



*The agent holds nothing; LiteLLM decides, signs and accounts. The agent sees a placeholder, OpenShell holds the agent's virtual key, LiteLLM holds its master key and the spend, and OCI knows the node's identity.*

All of it runs in one tenancy, and nothing on a laptop. An OKE Basic cluster with one Ampere A1 node runs the gateway, the classifier and Postgres. One Ampere A1 VM runs the control room, the OpenShell gateway and the agent's sandbox. An OCI API Gateway in front gives the control room an HTTPS address with an Oracle-issued hostname and certificate, and nothing else is reachable from the internet. The gateway signs every OCI call as its node, so no OCI key or token is stored in the cloud; the node's dynamic group needs three policy statements:

```
Allow dynamic-group litellmdemo-nodes to use generative-ai-family in compartment litellmdemo
Allow dynamic-group litellmdemo-nodes to manage generative-ai-response in compartment litellmdemo
Allow dynamic-group litellmdemo-nodes to read repos in compartment litellmdemo
```




*The whole demo in Oracle Cloud: OKE, one VM, an API gateway in front. Five idempotent scripts build it, one checks it from the outside, and one deletes everything the others created and nothing else.*

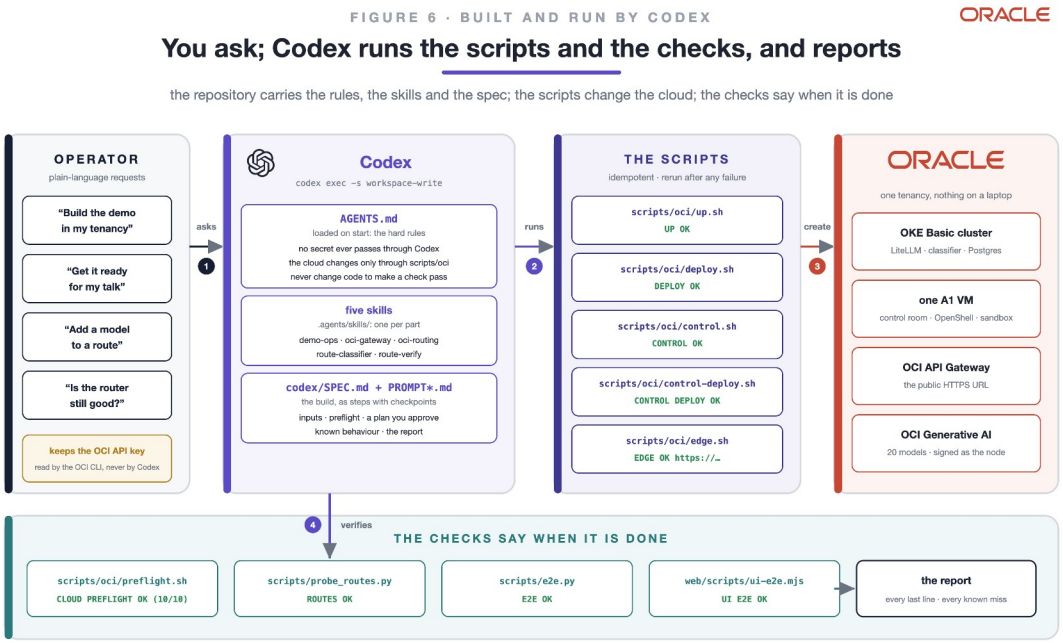
The node and the VM fit the Always Free Ampere allowance when a tenancy has it to spare; otherwise they are billed by the hour, the API gateway per million calls, and the models per token. All seventeen use cases together cost about $0.17 in model calls (`evidence/e2e-cloud-2026-10-13.txt`).

## Build: ask Codex to stand it up in your tenancy

You write one file, `~/.config/litellmdemo/deploy.json`, with your OCI CLI profile and regions, clone the repository, start Codex, and hand it the prompt that ships with it:

```
git clone https://github.com/fede-kamel/oci-litellm-routing && cd oci-litellm-routing
codex -s workspace-write -c sandbox_workspace_write.network_access=true \
  --add-dir ~/.config/litellmdemo --add-dir ~/.kube \
  --add-dir ~/.cache/uv --add-dir ~/.local/share/uv --add-dir ~/.npm \
  "$(cat codex/PROMPT.md)"
```

The flags let Codex reach the network and write where the demo keeps its state, kubectl its cache, and the package managers theirs. Codex reads `AGENTS.md`, which it loads on its own: no secret passes through it, the cloud changes only through `scripts/oci/*.sh` and only when the operator asks, no paid labelling runs unless asked, and no code changes to make a check pass. It reads the specification, `codex/SPEC.md`, and the skill for each part. It runs a preflight, shows the plan (what it will create, in which region, at which sizes, and what that costs) and waits for a yes. Then it runs the five build scripts in order, each ending with a line it checks (`UP OK`, `DEPLOY OK`, `CONTROL OK`, `CONTROL DEPLOY OK`, `EDGE OK https://…`), records one run of every use case for replay, runs the four checks, and reports. The OCI API key stays where the OCI CLI reads it; the gateway's keys stay in their 0600 files; Codex reads neither.




**You ask. Codex runs the scripts and the checks. The keys stay where the scripts read them.**

*You ask; Codex runs the scripts and the checks, and reports. The repository carries the rules, the skills and the specification; the scripts change the cloud; the checks say when it is done.*

What makes the specification worth trusting is that Codex attacked it before anyone followed it. A fresh Codex session, in a read-only sandbox and with nothing but the repository, read the build specification and the prompts as a stranger would and checked every command, flag and expected line against the scripts they describe. It found 35 defects (`evidence/codex/codex-spec-review-1-2026-10-15.md`), and four of them would have stopped a first build in an empty tenancy: the VM never installed OpenShell, because the first cloud build had done that step by hand; only the laptop path generated the gateway's master key; a progress line printed to standard output corrupted a subnet identifier on a fresh run; and the web front end's lockfile pointed at a private package mirror. All 35 were fixed in the scripts, the specification and the skills. A second fresh session then reviewed the fixed harness and found 13 more, nearly all about reruns after an interruption rather than the first build: a re-issued agent key that did not reach the sandbox's provider, an interrupted create that a rerun would not adopt, a first-boot step that could fail and still mark the VM ready. Those were fixed too (`evidence/codex/codex-spec-review-2-2026-10-15.md`), and a fresh clone now installs, builds and routes with the shipped model before any cloud call. Draft, adversarial review, fix, review again: the loop is cheap, and it is the difference between a demo that works for its author and one a customer can reproduce.

That is the honest state of the build path: reviewed twice by the agent that will execute it, all 48 findings fixed, and not yet run end to end in an empty tenancy since those fixes. The environment the demo runs in was built by the same scripts with the by-hand step the review found, and every check after it is in the evidence.

## Operate: ask Codex to get it ready for a talk

Once it exists, the everyday request is one sentence, and the prompt for it ships too (`codex/PROMPT-ops.md`): run the cloud preflight and fix what the scripts can fix, send three requests through the agent's sandbox, run the escape probe, and report. The three requests are chosen to exercise the two primaries that once failed quietly, GPT-5.3-codex on the Responses API and Cohere after a tool call, and the cheapest route. The probe asks the sandboxed agent to reach OCI directly, a model vendor, and the gateway's own admin API.

On 15 October Codex ran that prompt from a laptop on a network the cluster's allowlist does not admit, with one line of operator context: report only, change nothing. It is the more useful run to show. The cloud preflight came back 5 of 10: everything a visitor uses passed through the public URL (the API gateway deployment, the page, the OKE gateway as the primary, the sandbox and the agent's key within budget, a live stream), and every check that needs the cluster's API or SSH to the VM failed. Codex named the likely cause from the skill's table, an operator address the rules no longer admit, and the fix, `scripts/oci/up.sh` and `scripts/oci/control.sh`, and did not run it: that fix changes the cloud's allowlist and locks out the previous network, which is the operator's call. It also declined to record replays, because that is paid. Then it ran the rest through the public URL (`evidence/codex/codex-cloud-ops-2026-10-15.log`):

```
Request       Route         Model                        Calls  Fallbacks  Spend      Runtime
support-faq   SIMPLE        GPT-5.4-nano, GPT-6-luna     2      0          $0.000154  openshell
code-race     CODE          GPT-5.3-codex                1      0          $0.007308  openshell
lang-ja       MULTILINGUAL  Cohere Command A             2      0          $0.001042  openshell

Escape probe  OCI directly: REFUSED   api.openai.com: REFUSED   /key/generate: REFUSED (403)
Agent key     $1.47 of $5 this period
```

It checked that each run came from the sandbox, not the host agent or a replay, from the evidence files rather than the summary line, and it noted where its own evidence was thin: the probe's network list held one DNS denial for three refusals. The whole run took about 38,000 tokens.

## The safety net, measured

For a reviewer the question is never whether the demo works; it is what happens to everything it did not show. So the demo measures both sides from inside the sandbox. The grant is one endpoint: chat completions and the model list on the gateway, for the Python interpreter. The escape probe tests the default:

```
OCI Generative AI, directly           REFUSED: no connection
a model vendor (api.openai.com)       REFUSED: no connection
the gateway admin API (/key/generate) REFUSED by policy (403)
```

The first two are not specially forbidden; they have no rule, so OpenShell never opens the connection, and its log names the decision (`DENIED inference.generativeai.us-chicago-1.oci.oraclecloud.com [reason:policy_dns_ineligible]`). The third is the gateway's own host, reachable for chat and refused for administration. In the measured runs, 14 of 14 use cases ran entirely in the sandbox, and its only connections were 30 chat-completion requests to the gateway (`evidence/openshell/e2e.jsonl` in the repository). Under that profile the agent cannot raise its budget, mint a key, or call a model the gateway did not choose for it.

The second layer is LiteLLM's. The agent's virtual key may call five model groups, at 120 requests a minute, up to $5 every 30 days, and every call is charged to it; a visitor at the public URL cannot spend more than the key allows. The third is OCI's: the gateway signs as its node, and the node's policy names one compartment.

## What it is today

This is a working system in a real tenancy. The gateway, the classifier and Postgres run on OKE; the control room, OpenShell and the sandbox on one VM; the API gateway in front. After the fixes of 12 October the cloud preflight passed 10 of 10, the browser test 28 of 28, and the route probes 11 of 12, with GPT-5.3-codex on CODE and Cohere on MULTILINGUAL and no fallback; the miss is the router's known one (`evidence/cloud-run.md`). Every use case through the public URL passed 14 of 17, twice: two requests take the same known miss, and the incident job's arithmetic step missed its number in every run that day. Those are reported, not tuned away.

The Codex harness covers both ends: building it from a clone, and getting it ready for a talk. Codex as a client of the gateway passed every case from a fresh clone of the sample, on a gateway running the repository's configuration; the cloud gateway gets the same code, and the `/v1` route that lets Codex reach it with its own key, at its next deploy. What is not yet demonstrated is the first build of the fixed scripts in an empty tenancy, by Codex, end to end; it is the next run, and it needs billed resources and an operator's yes.

## Field notes from the build

Every lesson below cost real time. They are in the skills and the specification now, because a customer's first hour meets exactly these.

- **A fallback that works hides a primary that does not.** Signed as the OKE node, every GPT-5.3-codex call returned 404 "Authorization failed or requested resource not found" in a tenth of a second and fell back to GPT-5.1. Every check passed, because they judged the route, not the model. `generative-ai-family` covers OCI's chat API, not its Responses API; the node needs a `manage generative-ai-response` statement. The checks now fail a code or multilingual run that fell back.
- **Cohere after a tool call.** LiteLLM 1.104 sends the tool result in `chatHistory` and keeps `message`; OCI answers 400 "cannot specify message if the last entry in chat history contains tool results". The accepted shape is a top-level `toolResults` with an empty `message`, and then LiteLLM's citation type rejected OCI's `documentIds`. `gateway/oci_cohere_fix.py` corrects both; reported upstream as BerriAI/litellm#46157.
- **The complexity router and OCI's limits.** It sets `max_tokens` from LiteLLM's model map; OCI's Gemini accepts less than 65,536 and Llama 4 at most 4,096, so those deployments fell back silently until each declared `max_output_tokens`.
- **The LLM classifier's timeout.** Its default three seconds fell back to the heuristic without an error. Use the fastest model and set `timeout_ms`.
- **Agent requests drown the request.** An agent's system prompt and tool turns made every prompt look alike to the classifier, and Spanish read as English. Classify the user turns only.
- **OpenShell on Oracle Linux.** The UEK kernel builds Landlock in but leaves it off, and rootless Podman needs the `cpu` controller delegated to the user; `scripts/oci/control.sh` sets both at first boot and waits for the reboot.
- **API Gateway and streams.** OCI API Gateway closes a response that stays silent for 15 seconds; a reasoning model thinks longer. The control room's streams send a keep-alive every five seconds.
- **Codex through the gateway.** Streamed calls need an uncompressed stream from OCI (`Accept-Encoding: identity`); a custom provider needs a streaming method or every streamed turn falls back; the router must skip a coding agent's own context (its AGENTS.md and environment block

arrive as user turns); OCI's chat API refuses `reasoning_effort` with function tools on GPT-5.4 and Grok 4.20, so the gateway drops it on tool calls; and a coding agent's non-English request goes to the trained router, because Cohere Command A answered a 192 KB tool list with nothing.

- **Corporate networks.** Some refuse ports 22 and 6443 to cloud addresses whatever the allowlist says; the build scripts need a network that allows them.
- **The coding agent has its own constraints.** The ChatGPT-account Codex stopped accepting the model in its own configuration between two runs; pass `-m`. A headless `codex exec` cannot answer approval prompts, so everything a run needs must fit its sandbox. And a review is only as good as its rules: the read-only one found more than any test run had.

## Reproduce it

Every number above cites its transcript inline, and the kit's `evidence/` folder holds them all; the recordings are in `videos/`. None of it is a private setup. The sample repository, oci-litellm-routing, has a ten-minute quickstart: a gateway on your machine with an OCI API key, then Codex pointed at it, then `scripts/codex_e2e.py` to check it. For the whole system in your tenancy, hand Codex `codex/PROMPT.md` to build, or `codex/PROMPT-ops.md` before a talk; the commands and flags are in `codex/README.md`, and the specification's appendix of commands is what Codex runs underneath, for anyone who wants to audit or type it. Model calls cost cents; the node and the VM fit the Always Free allowance when it is available.

## The larger point

The demo is about routing, but two things in it are reusable. The first is measuring before routing: label by outcome, predict a chance of a good answer rather than a label, and let a gateway that already handles keys, budgets and fallbacks act on it, with its decision in every response. The second is the method around it: a specification with checkpoints, an adversarial review by the agent that will execute it, rules that keep secrets and cloud changes out of its hands, and checks that say when it is done. That pattern turns a careful piece of infrastructure into something a team can stand up by talking to a coding agent instead of learning four command lines, and it is small enough to ship with every architecture we publish.

You ask. Codex builds and checks. LiteLLM picks the model, and the agent holds nothing. Nothing in it asks to be believed; it asks to be re-run.

## References

- **The sample repository**: the gateway, the router, Codex as a client, the agent and its sandbox, the control room, the cloud scripts, the Codex harness and offline tests: oci-litellm-routing.
- **LiteLLM on OCI**: the provider guide and the announcement, LiteLLM natively supports OCI Generative AI.
- **OCI Generative AI**: documentation.
- **NVIDIA OpenShell**: repository.
- **langchain-oci**: oracle/langchain-oracle.
- **OpenAI Codex**: documentation, including `AGENTS.md`, skills, and sandbox configuration.

## Appendix — the upstream work behind this

| Where | What | Status | Role here |
|---|---|---|---|
| BerriAI/litellm#28223 | OCI Generative AI as a first-class LiteLLM provider: every vendor family, tools, vision, reasoning, embeddings, | Merged | The provider every route here uses |

| Where | What | Status | Role here |
|---|---|---|---|
| | instance principal and OKE workload identity | | |
| BerriAI/litellm#46157 | Cohere on OCI after a tool call: the request shape and the citation field | Reported | Why `gateway/oci_cohere_fix.py` exists |
| NVIDIA/OpenShell#3904 | the `oci-genai` provider profile | Merged | The provider-profile pattern the sandbox's LiteLLM profile follows |

*The demo, its Codex harness and this write-up are personal open-source work, built and tested with personal resources. Northwind Cloud is a fictional company and no real customer data is used; this is not an Oracle publication or a statement of Oracle's plans.*